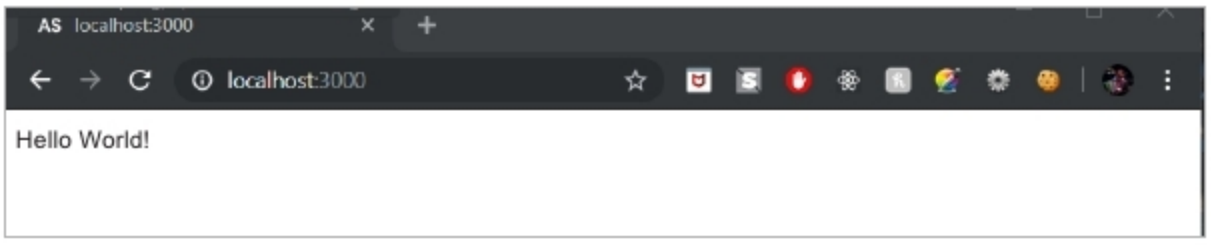

Figure 6: Test run on the browser

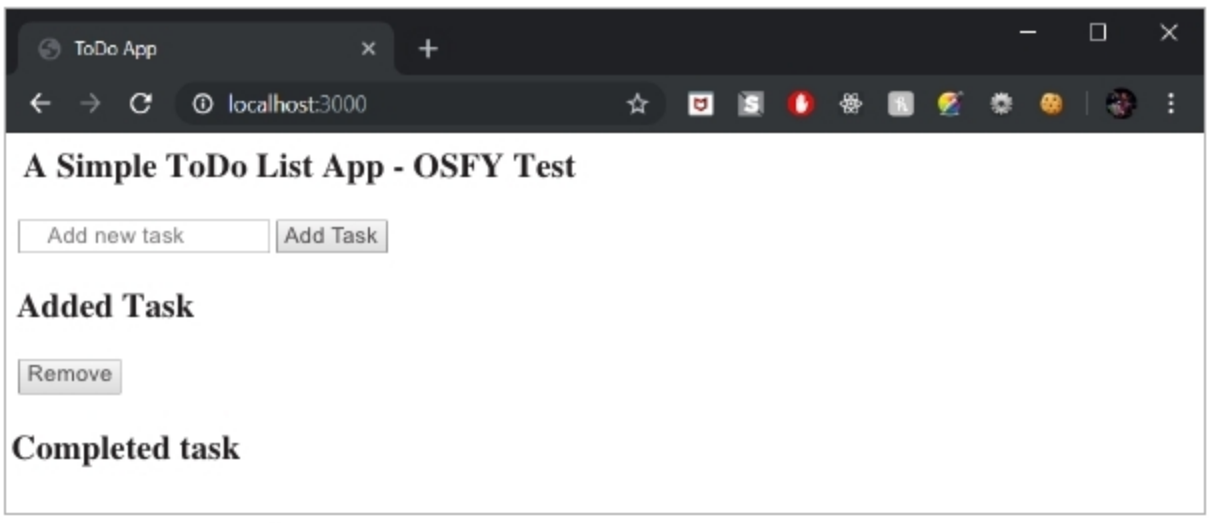

Figure 7: The *index.ejs* file

your console:

```
node index.js
```

To check the output, open your favourite browser and visit *localhost:3000*. This will show us the response that it was meant to show – *Hello World!*

In a real-case scenario, we do not want to respond with Hello World, do we? So, we would prefer to definitely send back an HTML file with all that we want to display to our users – it could be forms, images, videos, and the list goes on. Since we are creating a to-do list app, we will have a form for adding new tasks, two buttons to add and remove items, and some basic text. For that, we will be using EJS (Embedded JavaScript), a templating language. Install EJS by running the following command:

```
npm install ejs --save
```

Now, we will set up the template engine with the following line of code in the *index.js* file:

```
app.set('view engine', 'ejs');
```

By default, EJS is accessed in the views directory of the app. So, we will create a new folder named *views* in our directory and inside this folder, add a new file named *index.ejs*. Consider this as our HTML file:

```
<html>
<head>
<title> ToDo App </title>
 <link href="https://fonts.
googleapis.com/css?family=Lato:100"
rel="stylesheet">
 <link href="/styles.css"
rel="stylesheet">
</head>

<body>
<div class="container">
<h2> A Simple ToDo List App - OSFY
Test</h2>

<form action ="/addtask" method="POST">

<input type="text" name="newtask"
placeholder="add new task">
<button> Add Task </button>


<h2> Added Task </h2>

<button formaction="/removetask"
type="submit" id="top"> Remove </
button>
</form>

<h2> Completed task </h2>
</div>

</body>
</html>
```

In order to display this, let us replace our *app.get* code in the *index.js* file so that it sends the corresponding HTML to the client.

```
app.get('/', function(req, res){
    res.render('index');
});
```

Now, when we run *node index.js* and navigate to *localhost:3000* in our browser, you should be able to see the *index.ejs* file being displayed.

So we have set up a route for our app. But, for our to-do app to work, we need a post route as well. In our *index.ejs* file, our form is submitting a post request to the */addtask* route:

```
<form action ="/addtask" method="POST">
```

This is where our form is posting data, so we will set up the route now. The *post* request looks similar to a *get* request, but with certain changes:

```
app.post('/addtask', function (req,
res) {

 res.render('index')
});
```

In this case, we cannot just respond with the same HTML template, but have to access the name of the newtask typed in by the user. For this, we will have to use some Express middleware – functions that have access to the *req* and *res* bodies in order to perform more advanced tasks. We will use the body-parser middleware. It allows you to use the key-value pairs stored on the